

Curso de Go

Aula 10 - Select e buffered channels em Go

Professor : Antônio Marcelo

NÚCLEA
ACADEMY

 **C/B** [código_] **BRAZUCA** [3]





Nessa Aula nos Vamos Ver

- **Nesta aula iremos aprofundar os mecanismos de comunicação concorrente da linguagem Go utilizando:**
 - **select**
 - **buffered channels**
 - **sincronização entre goroutines**
- **Também veremos problemas clássicos de concorrência, como:**
 - **bloqueios**
 - **starvation**
 - **deadlocks**
- **Parte Prática**



Revisão rápida de channels

- **Channels são mecanismos de comunicação entre goroutines.**
- **Eles permitem:**
 - **envio de dados**
 - **sincronização**
 - **troca segura de informações**
- **Característica importante de Channels comuns:**
 - **bloqueiam envio**
 - **bloqueiam leitura**
 - **Até existir comunicação entre as partes.**



O que é multiplexação

- **Conceito**
- **Multiplexação significa:**
 - **esperar múltiplas fontes de dados**
 - **responder ao primeiro evento disponível**
 - **Em Go isso é feito com select**
- **Problema sem select**
- **Imagine:**
 - **vários channels**
 - **várias goroutines**
 - **múltiplos eventos**
- **Sem select, controlar isso fica complexo.**



Introdução ao select

- **O select:**
 - **espera múltiplos canais**
 - **executa o primeiro disponível**

Sintaxe:

```
select {  
case msg := <-canal1:  
    fmt.Println(msg)  
  
case msg := <-canal2:  
    fmt.Println(msg)  
}
```




Introdução ao select

- exemplo completo
- Resultado
- O primeiro canal disponível será processado.

```
package main

import (
    "fmt"
    "time"
)

func canal1(ch chan string) {
    time.Sleep(1 * time.Second)
    ch <- "Resposta canal 1"
}

func canal2(ch chan string) {
    time.Sleep(2 * time.Second)
    ch <- "Resposta canal 2"
}

func main() {
    ch1 := make(chan string)
    ch2 := make(chan string)

    go canal1(ch1)
    go canal2(ch2)

    select {
    case msg := <-ch1:
        fmt.Println(msg)

    case msg := <-ch2:
        fmt.Println(msg)
    }
}
```



Select com default

- O bloco default executa quando nenhum channel está pronto

```
select {  
  case msg := <-ch:  
    fmt.Println(msg)  
  
  default:  
    fmt.Println("Nenhum dado disponível")  
}
```




Select com default

Exemplo Completo:

```
package main

import "fmt"

func main() {
    ch := make(chan int)

    select {
    case valor := <-ch:
        fmt.Println(valor)

    default:
        fmt.Println("Channel vazio")
    }
}
```

- **Uso prático**
- **Muito usado para:**
 - **polling**
 - **verificações rápidas**
 - **evitar bloqueios**



Timeout com select

- **Problema**
- **Às vezes uma goroutine:**
 - **demora demais**
 - **trava**
 - **nunca responde**
- **Precisamos controlar timeout.**
- **Solução**
 - **Usar:**
 - **time.After()**
 - **junto com select**



Timeout com select

- **Resultado : O timeout ocorre antes da resposta.**

Exemplo Completo:

```
package main
```

```
import (  
    "fmt"  
    "time"  
)
```

```
func tarefa(ch chan string) {  
    time.Sleep(3 * time.Second)  
    ch <- "Concluído"  
}
```

```
func main() {  
    ch := make(chan string)
```

```
    go tarefa(ch)
```

```
    select {  
    case msg := <-ch:  
        fmt.Println(msg)
```

```
    case <-time.After(1 * time.Second) :  
        fmt.Println("Timeout")  
    }  
}
```



Buffered channels

- **O que são buffered channels ?**
 - **Channels bufferizados armazenam valores temporariamente.**
- **Diferença**
- **Channel comum**
 - **Bloqueia imediatamente.**
- **Buffered channel**
 - **Permite alguns envios sem receptor imediato**
- **Vantagem**
 - **Reduz bloqueios temporários.**



Buffered channels

Exemplo:

```
package main
```

```
import "fmt"
```

```
func main() {  
    ch := make(chan int, 2)
```

```
    ch <- 10
```

```
    ch <- 20
```

```
    fmt.Println(<-ch)
```

```
    fmt.Println(<-ch)
```

```
}
```



Buffered channels na prática

- **Cenário real**
- **Imagine:**
 - fila de processamento
 - múltiplos workers
 - mensagens chegando rapidamente
- **Buffered channels funcionam como:**
 - filas concorrentes

Explicação

- **O buffer:**
 - permite enfileirar tarefas
 - sem bloquear imediatamente

```
package main

import (
    "fmt"
    "time"
)

func worker(ch chan int) {
    for valor := range ch {
        fmt.Println("Processando", valor)

        time.Sleep(time.Second)
    }
}

func main() {
    ch := make(chan int, 5)

    go worker(ch)

    for i := 1; i <= 5; i++ {
        ch <- i
    }

    close(ch)

    time.Sleep(6 * time.Second)
}
```




O que é deadlock

- **Deadlock ocorre quando:**
 - **todas as goroutines ficam bloqueadas**
 - **nenhuma consegue continuar**

Exemplo:

```
package main

func main() {
    ch := make(chan int)

    ch <- 10
}
```

- **Problema**
 - **Não existe ninguém lendo o channel.**
 - **A escrita bloqueia para sempre.**



Problemas comuns em concorrência

- **Deadlocks**
- **Causas:**
 - envio sem receptor
 - leitura sem envio
 - goroutines travadas
- **Race conditions**
 - Múltiplas goroutines alterando a mesma variável
- **Starvation**
 - Uma goroutine nunca consegue executar
- **Vazamento de goroutines**
 - Goroutines ficam:
 - presas
 - aguardando eternamente



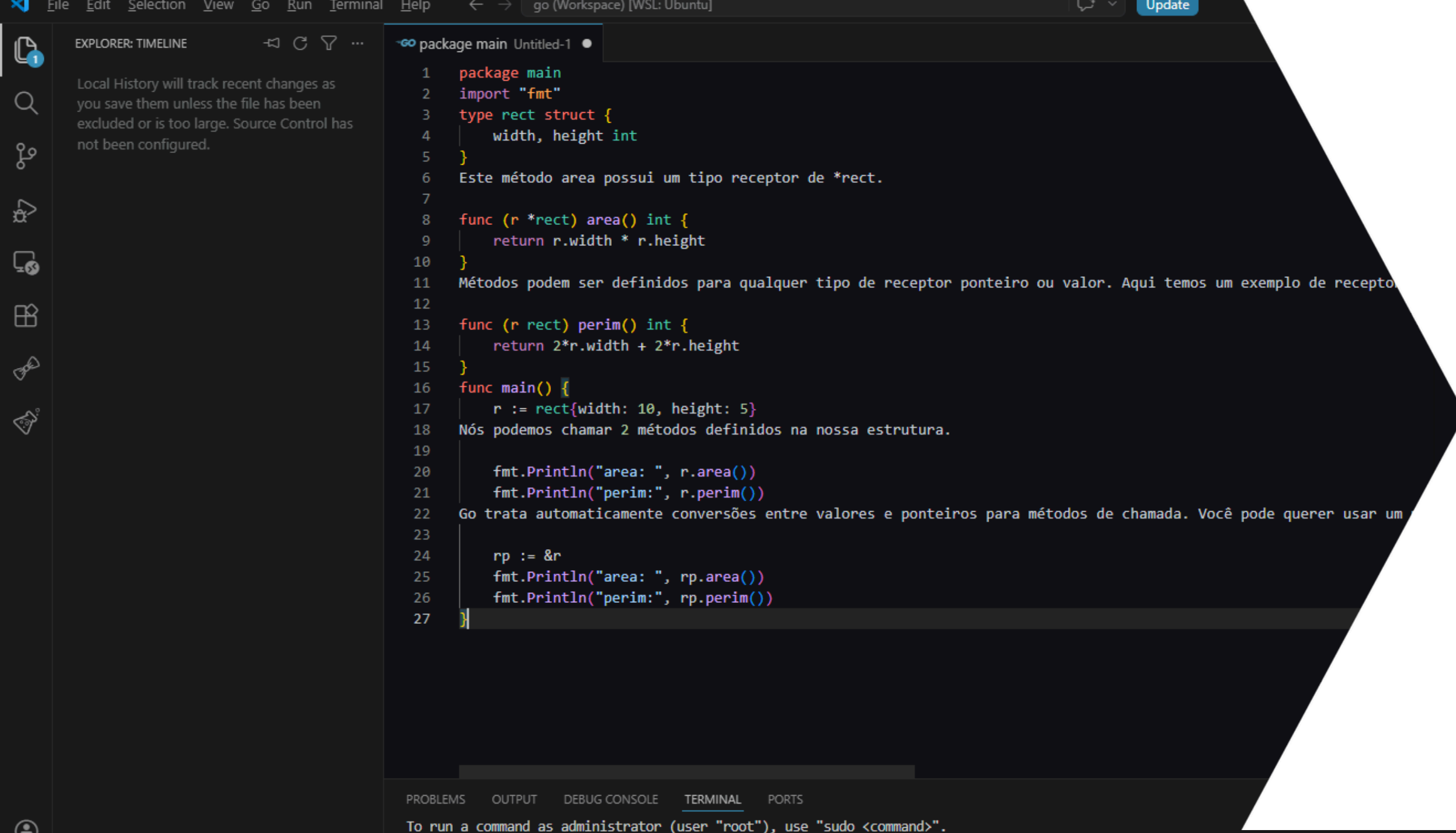
Boas Práticas

- **Prefira comunicação segura, Use:**
 - **channels**
 - **select**
 - **sincronização clara**
- **Feche channels corretamente**
 - **close(ch)**
- **Evite loops infinitos sem controle**
 - **Ruim:**

```
for {  
      
}
```
- **Utilize timeout sempre que possível**

```
time.After()
```
- **Monitore concorrência**
- **Ferramenta útil:**

```
go run -race main.go
```



The screenshot shows a Go IDE with a package main. The code defines a struct `rect` with `width` and `height` of type `int`. It includes two functions: `area()` which returns `r.width * r.height`, and `perim()` which returns `2*r.width + 2*r.height`. The `main()` function creates a `rect` struct with `width: 10` and `height: 5`, prints its area and perimeter, and then passes a pointer to the struct (`&r`) to the same functions to demonstrate pointer handling. Comments in Portuguese explain the purpose of the functions and the pointer usage.

```
1 package main
2 import "fmt"
3 type rect struct {
4     width, height int
5 }
6 Este método area possui um tipo receptor de *rect.
7
8 func (r *rect) area() int {
9     return r.width * r.height
10 }
11 Métodos podem ser definidos para qualquer tipo de receptor ponteiro ou valor. Aqui temos um exemplo de recepto
12
13 func (r rect) perim() int {
14     return 2*r.width + 2*r.height
15 }
16 func main() {
17     r := rect{width: 10, height: 5}
18     Nós podemos chamar 2 métodos definidos na nossa estrutura.
19
20     fmt.Println("area: ", r.area())
21     fmt.Println("perim:", r.perim())
22     Go trata automaticamente conversões entre valores e ponteiros para métodos de chamada. Você pode querer usar um
23
24     rp := &r
25     fmt.Println("area: ", rp.area())
26     fmt.Println("perim:", rp.perim())
27 }
```

NÚCLEA
ACADEMY

Parte Prática

Praticar Select e buffered channels em Go